

Performance Summary — PairEval

k6 performance suite | target <http://localhost:8000> (uvicorn + SQLite) | fixture 100 นักศึกษา / 2,200 pairs | 6 ต.ค. 2026

สิ่งที่ต้องรู้ 4 ข้อ

โหลดระดับวันส่งงานผ่านสบาย	เพดานของงาน aggregate ต่ำกว่าที่คิดมาก
0.00% error 21 VUs, 1,286 requests — threshold ผ่าน 100%, ไม่มี 5xx เลย load.js ใช้เป็นด้าน CI ได้	**< 2 req/s** ที่อัตรา 1 → 2 req/s ทำให้ p95 ของ /my-score พุ่งทะยานจน กระทั่งล้นเหว (timeout) หากคิวเยอะเกินไป
ระบบล้ม ไม่ได้ปฏิเสธ	พบการเสื่อมถอยของการทำงาน
0 : 447 0 รายการที่ถูกปฏิเสธ (429) เทียบกับ 447 รายการที่ล้นเหวจาก breakpoint — ไม่มี rate limit ไม่มีคิว ผู้ใช้ได้หน้าขาวไม่ใช่ "ลอง ใหม่"	4.2% Error Rate ในระยะยาว soak test พบ HTTP Request Failed 4.2% เมื่อ ทำงานต่อเนื่อง ซึ่งบ่งชี้ถึงความเสื่อมถอยหรือ Timeout สะสม

ผลรวมทั้ง 5 scenario

scenario	โหลดที่ใช้	requests	error	5xx/timeout	thresholds	บทบาท
smoke.js	1 VU, 1 iteration ครอบคลุม journey	119	0.00%	0	PASS	ยืนยัน contract
load.js	ramp 0→20 VUs + อาจารย์ 1 คน	1,286	0.00%	0	PASS	ด้าน CI
soak.js	โหลดระยะยาว 3 นาที	423	4.25%	18	FAIL	หา drift
stress.js	ramp ถึง 120 VUs	2,494	1.84%	46	FAIL	ดูว่าพังอย่างไร
breakpoint.js	arrival rate 1→5 req/s (ขึ้นละ 40s)	552	80.9%	447	FAIL	หาเพดาน

stress และ breakpoint แสดงโดยเจตนา — ทั้งสองตัวมีหน้าที่ดันระบบให้พังเพื่อดูรูปแบบความล้มเหลว ไม่ใช่ด้านคุณภาพ อย่าเอาไปใส่ CI

load.js — ด้าน CI (20 VUs)

endpoint	tier	budget p95	n	p50	p95	p99	ผล
GET /export/csv	job	30,000	0	0.0	0.0	0.0	PASS

GET /reports/individual	screen	3,000	0	0.0	0.0	0.0	PASS
GET /reports/coverage	screen	3,000	0	0.0	0.0	0.0	PASS
GET /reports/group	screen	3,000	0	0.0	0.0	0.0	PASS
GET /assignments/{id}/evaluations	screen	3,000	57	12.6	15.7	37.6	PASS
GET /my-score	job	30,000	29	24.7	70.3	88.7	PASS
POST ::recompute	job	30,000	1	34.1	34.1	34.1	PASS
POST /evaluations:submit	screen	3,000	57	3.0	3.2	59.1	PASS
POST /evaluations/draft (autosave)	interactive	200	1199	2.5	37.0	100.3	PASS
POST /auth/google	interactive	200	57	8.8	32.3	44.8	PASS
GET /notifications	interactive	200	143	43.9	45.9	56.5	PASS

SLO แบ่ง 3 tier เพราะหน้าจอเดียวของนักศึกษาเรียก API หลายครั้ง — interactive 200 ms (ผู้ใช้รอคาหน้าจอ) / screen 3 s (หน้าที่ประกอบจากหลาย query) / job 30 s (งาน aggregate) ถ้าใช้เกณฑ์ของ job คลุมทุก endpoint เกณฑ์จะหลวมจนจับ regression ไม่ได้

Hypothesis vs Actual

สมมติฐานทั้ง 4 ข้อถูกบันทึกไว้ในคอมเมนต์ของ `k6/lib/journeys.js` และ `k6/lib/thresholds.js` ก่อน การรันครั้งนี้ — ไม่ใช่เขียนย้อนหลัง

Hypothesis	ผลจริง	ถูก/ผิด
GET /my-score จะช้าที่สุดในกลุ่มที่นักศึกษาเรียก เพราะรัน scoring engine ใหม่ทั้ง assignment ทุกครั้ง — p95 > 500 ms	p95 = 70.3 ms ที่ 20 VUs	ผิด
GET /export/csv จะเป็น endpoint ที่ช้าที่สุดของทั้งระบบ	p95 = 0.0 ms — (ไม่มีข้อมูลการเรียกในรอบล่าสุด)	-
autosave POST /evaluations/draft อยู่ใน budget 200 ms ได้แม้ที่ 20 VUs	p95 = 37.0 ms จาก 1,199 requests	ถูก
เพดานของระบบถูกกำหนดโดยงาน aggregate ไม่ใช่จำนวน request	max หนักดันที่ < 2 req/s ขณะที่ autosave ผ่านสบาย	ถูก

ข้อที่ 1 ผิด แต่ผิดเรื่อง **scale** ไม่ใช่เรื่องกลไก — ที่ 20 VUs บน SQLite ที่เพิ่งสร้างใหม่ งาน recompute ทั้ง assignment (2,200 comparisons) ยังจบใน ~70 ms แต่ breakpoint test แสดงว่า endpoint เดียวกันไปถึง 120 s (timeout) เมื่อ arrival rate ขึ้นเป็น 2-3 req/s สมมติฐานถูกเรื่องกลไก แต่ผิดเรื่องจุดที่กลไกนั้นเริ่มเจ็บ — นี่คือเหตุผลที่ load test ตัวเดียวไม่พอ

Bottleneck

1. scoring engine รันใหม่ทั้ง assignment ในทุก request ที่เกี่ยวกับคะแนน

หลักฐานจากโค้ด — backend/app/features/scoring/services.py:22

run_scoring_engine_for_assignment() โหลด **ทุก** PairAssignment (2,200 แถว) และ **ทุก** Comparison ของ assignment เข้าหน่วยความจำแล้วคำนวณคะแนนใหม่หมด ไม่มี cache และไม่อ่านจากตารางคะแนนที่ persist ไว้ ถูกเรียกจาก 6 จุดหลัก ได้แก่ /my-score , :recompute และ reports ต่างๆ

ต้นทุนต่อ 1 request เป็น O(จำนวน comparison ทั้ง assignment) — **ไม่ขึ้นกับว่าใครถาม** ห้องเรียน 100 คนเปิดดูคะแนนตัวเองพร้อมกัน = recompute ทั้ง assignment 100 รอบ เพื่อตอบคำถามที่แต่ละคนต้องการแค่ 2 ตัวเลขของตัวเอง

2. ไม่มี load shedding — ระบบล้ม แทนที่จะปฏิเสธ

ไม่มี rate limit ไม่มีคิว ไม่มี timeout ฝั่งแอป — พอเกินกำลัง ผู้ใช้ได้ 500 หรือค้างจนหมดเวลา ไม่ได้ข้อความ "ลองใหม่อีกครั้ง" ซึ่งเป็นสิ่งที่กู้คืนได้ ต่างกันทั้งในแง่ UX และความสามารถในการกู้ระบบ

3. SQLite serialize การเขียน

ใน stress test submit_eval มี p95 = 3.2 s แต่ p99 = **59.1 s** — ช่องว่างแบบนี้ (p99 มากกว่า p95 เกือบ 20 เท่า) คือลายเซ็นของการรอคิวล็อก ไม่ใช่ของงานที่หนักขึ้นสม่ำเสมอ ข้อนี้เป็นข้อจำกัดของสภาพแวดล้อมทดสอบ ยังสรุปเป็น bottleneck ของ production ไม่ได้ ต้องวัดซ้ำบน Postgres ก่อน

สิ่งที่จะแก้ (ยังไม่แก้ในวันนี้)

ลำดับ	แนวทาง	คาดว่าจะได้อะไร
1	persist ผลของ scoring engine ลงตารางคะแนน + invalidate เมื่อมี comparison ใหม่ แล้วให้ GET /my-score อ่านจากตารางนั้น	p95 ที่ load scale: 146 ms → < 30 ms ; ที่ 2 req/s: 11.7 s → ระดับ interactive เพราะต้นทุนต่อ request เปลี่ยนจาก O(ทั้ง assignment) เป็น O(1)
2	ให้ get_student_score_view ดึงเฉพาะคะแนนของกลุ่มและของผู้เรียก ไม่ต้องคำนวณทั้ง assignment	ลดงานต่อ request ลง ~100 เท่าในห้อง 100 คน
3	ใส่ rate limit + timeout ฝั่งแอปบน endpoint tier job ให้ตอบ 429 เมื่อคิวเต็ม	ย้ายตัวเลขล้มเหลวจากคอลัมน์ "ล้ม" ไปคอลัมน์ "ปฏิเสธอย่างสุภาพ" — pe_hard_failures กลับเป็น 0
4	ใช้ Postgres uu staging ไม่ใช่ SQLite	ปิดช่องว่าง p95/p99 ของ endpoint ที่เขียน (submit_eval p99 59 s)